

ZeroDay

A desk that reads a shipping mailbox, decides what each email is asking for, and checks a draft bill of lading against the shipping instruction.

ZeroDay is a working clerk console for the Averis shipping-document problem. Staff lose time finding the right email, and they lose more time comparing two documents by hand. Names, ports, quantities, and weight must match, while the same fact is often written under a different label. A request that is never opened never reaches that check. A mismatch that is missed becomes a correction, a delay, and another round of email.

The prototype takes a mailbox as a zip, or over IMAP. It classifies every message, compares the shipping instruction with the draft bill of lading when that is the job, and escalates when it cannot decide. A clerk accepts the instruction, the bill, or a typed correction. Every program decision and every human change is written to an audit log. The desk is public on Cloud Run, the data lives in Firestore, and the live model is Gemini 3 Flash on Vertex AI.

Open it. <https://zeroday-316196081380.asia-southeast1.run.app> — clerk@zeroday.local / clerk123.

Source: <https://github.com/Dkkama/ZeroDay>

1. The problem

The user is a shipping-documentation clerk. The inbox mixes several kinds of work, and only one of them is a document check.

- Check the documents. The writer has attached a shipping instruction and a draft bill of lading and wants them compared. The instruction is the source of truth.
- Issue a shipping instruction. The writer is sending instructions now and may ask for a draft bill later. That later sentence is not a compare job. There is often no attachment.
- Answer an invoice question. Freight, local charges, detention, a missing goods receipt, a cancelled invoice.
- Operational mail and spam. Berthing reports, reminders, holiday notices, and junk. These must not enter the document queue.

Manual comparison fails in a specific way. One document says “Port of Loading” and the other says “Load Port” or “POL”. One says “To the order of” and the other says “Consignee”. “6 x 40HC” and “6 x 20GP” are the same container count. “APRIL FAR EAST” and “APRIL FINE PAPER TRADING” are not the same shipper. A blank, a line of question marks, or a commercial invoice saved under a bill-of-lading filename is uncertainty, not a mismatch. Guessing a value there is worse than sending the case to a person.

The preliminary prototype has to show that this loop works on the real 520-email set, not on a handful of hand-picked examples.

2. Technical architecture

The desk is one service. The browser never holds a model key and never talks to Firestore. Cloud Run serves the React application and the API on the same origin. Firestore is the system of record, because a JSON file on a Cloud Run instance disappears when the instance shuts down and cannot be shared by two clerks.

Piece	Role
Clerk desk	React interface: Dashboard, Inbox, Review, Audit, Settings.
API	FastAPI on Cloud Run. One public URL serves the desk and /api.
Database	Firestore in asia-southeast1. One document per email, audit row, and job.
Production model	Vertex AI, gemini-3-flash-preview. Three prompts at once, eight emails each.
Model selection	Cursor cloud agents, used to test models. Not on the request path a clerk uses.
Mail in	Zip of the inbox, or IMAP. Classification starts when the zip is dropped.

Path of one email

- A zip is dropped on the Dashboard or the Inbox, or IMAP fetches new messages. The zip must contain inbox JSON plus an attachments folder. macOS archive junk is ignored.
- Each attachment is turned into text. PDFs, spreadsheets, and plain text are handled. A file with no readable text is marked unreadable rather than filled in.
- The selected model receives up to eight emails in one prompt, with the classification rules and an instruction not to copy a value from one case to another. Three such prompts run at once, capped at five prompts a minute.
- The result is a category, a status (ok, mismatch, or needs review), the mismatched field names, and the seven field values from each document.
- Firestore stores the email, and an audit row records that the program classified it. The Inbox and the Review queue read that document.
- The clerk opens a review, picks the instruction value, the bill value, or types one, then validates. The screen moves to the next case immediately. The write follows. The audit row records the person, the field, and the time.

Cursor is a separate path used while we were choosing the model. It keeps three cloud machines warm and sends the same eight-email prompt. It is not required to run the public desk. Vertex calls Gemini directly with the Cloud Run service account, in the same Google project as Firestore and Cloud Run (hackathon-2026-509207, region asia-southeast1).

3. Implementation

The comparison rules live in one prompt, `sdoc_eval/prompts/v1.md`, shared by the evaluation harness and the desk. That is deliberate. A rule we could not score was a rule we did not trust in the product.

Categories and the compare

Five categories: document check, shipping-instruction request, invoice question, general operations, and spam. A document check is compared only when both files are present, readable, and actually an

instruction and a draft bill. Otherwise the status is needs-review, with a reason: wrong document type, missing attachment, unreadable, or missing value. A work order that only asks someone to send a draft which does not exist yet stays a document check with status ok. It is not an error and it is not an instruction request.

Seven fields are compared: shipper, consignee, notify party, port of loading, port of discharge, container count, and gross weight. Vessel, voyage, commodity, and booking number are ignored because they are not the scored decision. Party names are compared without the address block. Ports are compared on the city, so a UN/LOCODE in parentheses does not create a false mismatch. Container size is ignored.

Weight is the document total, in kilograms, with commas and the unit stripped. **How**

Vertex is called

One email is one Gemini request. Two to eight emails share one request. Nine emails are one request of eight and a second request for the remainder. A full inbox is the same rule repeated, three requests in flight, five per minute. The limit in production is 2048 emails, which is headroom rather than a product limit. If Google returns a rate-limit error, that batch waits and is tried again. An email missing from the model's JSON is retried. It is not filled in with a default category.

The clerk interface

Dashboard shows volume and how many document checks need a person. Inbox lists every message, with the preview opening under the row. Review is the queue of mismatches and escalations. Clicking the instruction or the bill updates the screen at once and saves behind that click, so the clerk is not waiting on Firestore between fields. Validate marks the row fixed and opens the next one. Audit can be exported. Settings holds the model switch, the zip drop zone, IMAP, and the job list.

Cloud Run is configured with 2 vCPU, 2 GB of memory, and a public URL. It scales to zero when nobody is using it. A classification of hundreds of emails is longer than one web request, so the full Vertex run was executed against the same Firestore database the site reads. Judges opening the inbox are looking at those stored decisions, not at a slide.

These sketches are the wireframes we drew while designing the desk, before the prototype was built, they show the progression of our project.



Inbox

Searches by subject, id, etc

Allows to sort by category of email

Button allows to export different types of emails in csv/json/xlsx formats

ZeroDay

- Dashboard
- Inbox
- Review request
- Audit log

Settings
Sign out

Inbox

Search for...

Category v Status v

Export

#	id	Subject	Date	Category	Status
1	email_001	TD CONFIRM DOCS_SR...	2026-01-01	General	OK
2	email_002	~~~~~	2026-01-01	General	OK
3	email_003	~~~~~	2026-01-01	BH-comparison	Needs Review
4	email_004	~~~~~	2026-01-01	General	OK
5	email_005	~~~~~	2026-01-01	General	OK
6	email_006	~~~~~	2026-01-02	General	OK
7	email_007	~~~~~	2026-01-02	Spam	OK
8	email_008	~~~~~	2026-01-03	SI-request	OK
9	email_009	~~~~~	2026-01-08	General	OK
10	email_010	~~~~~	2026-01-09	General	OK
11	email_011	~~~~~	2026-01-13	General	OK
12	email_012	~~~~~	2026-01-20	General	OK

Allows to sort by OK/Mismatch/Needs_Review

When clicked a preview appears with the SI and BH document

Review requests

ZeroDay

- Dashboard
- Inbox
- Review request
- Audit log

Settings
Sign out

Review requests

Click a row to review it.

#	id	Subject	Date caught	Reason
1	email_001	TD CONFIRM DOCS_SR...	2026-01-01	missing value
2	email_002	~~~~~	2026-01-01	unreadable
3	email_003	~~~~~	2026-01-01	missing value
4	email_004	~~~~~	2026-01-01	missing value
5	email_005	~~~~~	2026-01-01	missing attachment
6	email_006	~~~~~	2026-01-02	mismatch-consignee
7	email_007	~~~~~	2026-01-02	wrong doc type
8	email_008	~~~~~	2026-01-03	unreadable
9	email_009	~~~~~	2026-01-08	missing value
10	email_010	~~~~~	2026-01-09	missing attachment
11	email_011	~~~~~	2026-01-13	mismatch-shipper
12	email_012	~~~~~	2026-01-20	missing value

When clicked, sorts by number/letters in asc/des order

When clicked, a full page review is opened

Review requests

ZeroDay

- Dashboard
- Inbox
- Review request
- Audit log
- Settings
- Sign out

Review requests

[To List](#)
[Edit](#)
[Validate](#)

< >

Shipper
 SI: Shipper1 BL: Shipper2

Consignee
 SI: A BL: B

Notify party
 SI: ~~~ BL: ~~~

Port of loading
 SI: ~~~

Port of discharge
 SI: ~~~ BL: ~~~

Container count
 SI: ~~~ BL: ~~~

Gross weight
 SI: 123 BL: 321

Shipping instruction
 Shipper Shipper1
 Consignee A
 Notify party
 Port of loading
 Port of discharge
 Container count
 Gross weight 123

BL of lading
 Shipper Shipper2
 Consignee B
 Notify party
 Port of loading
 Port of discharge
 Container count
 Gross weight 321

Audit log

ZeroDay

- Dashboard
- Inbox
- Review request
- Audit log
- Settings
- Sign out

Audit log

[Export report](#)

#	id	Date	Actor	Change	Category
1	email_001	2026-01-01	program	Classified to...	General
2	email_002	2026-01-01	program	~~~~~	General
3	email_003	2026-01-01	human	~~~~~	BL-comparison
4	email_004	2026-01-01	program	~~~~~	General
5	email_005	2026-01-01	program	~~~~~	General
6	email_006	2026-01-02	program	~~~~~	General
7	email_007	2026-01-02	program	~~~~~	Spam
8	email_008	2026-01-03	program	~~~~~	SI-request
9	email_009	2026-01-08	program	~~~~~	General
10	email_010	2026-01-09	human	~~~~~	BL-comparison
11	email_011	2026-01-13	program	~~~~~	General
12	email_012	2026-01-20	program	~~~~~	General

Button allows to export last 50-50 document changes in csv/json/xlsx/pdf/txt formats

Allows navigation

Current page

4. What we measured

The official score weights category quality at 30 percent, defect detection at 20 percent, and an end-to-end check at 50 percent. End-to-end success means a planted defect was both routed to a document check and flagged on exactly the right fields. We ran that scorer twice.

Run	Final	Category	Planted defects	Escalations
Model selection gemini-3flash, 520 emails	1.0000	1.0000	46 / 46	20 / 20
Live desk on Vertex same 520 emails	1.0000	1.0000	46 / 46	20 / 20

The selection run is why the live model is Gemini 3 Flash. We cared about the balance of accuracy and cost on this mailbox, not about a general leaderboard. Category accuracy on that run was perfect across 220 document checks, 125 instruction requests, 75 invoice questions, 60 operational messages, and 40 spam messages. All 20 cases that should be escalated were escalated, including wrong document type, missing files, unreadable files, and missing values. All 46 planted defects were exact.

The live Vertex pass used the production calling pattern, not the evaluation machines. Category accuracy was again perfect, all 46 planted defects were exact, and all 20 escalations were caught. The final score is 1.0000.

We also checked the calling pattern itself. One email returns a single decision. Nine emails return eight decisions from the first prompt and the ninth from the leftover prompt. The 520-email inbox is the table above.

5. Challenges

Choosing the model

We spent a long time here because the model is the product. We already had a Cursor subscription, so the first design sent every email through Cursor. That was the wrong shape. Each call started a fresh virtual machine. The typical email took about a minute, and some calls died at the three-minute limit. Cursor is a good place to run a controlled test. It is a poor place to boot a computer every time a clerk receives mail.

We kept Cursor for the test harness: three machines stayed warm, eight emails rode in each follow-up, and we could compare models on the full inbox without rewriting the prompt. Gemini 3 Flash was the best balance of score and cost. The prototype then calls that model on Vertex, paid for with Google Cloud credits, with no virtual machine in the path.

Choosing where it runs

We did not have a hosting account we could keep on for free. After the first workshop we found that Cloud Run is covered by the Google Cloud trial credits. That settled the rest of the stack. Vertex, Firestore, and the website sit in one project, and the Cloud Run service account is allowed to call the model and the database. We did not want a second vendor for the database once the documents turned out to be a few kilobytes each.

Understanding the work

The problem statement is a clerk's job, and none of us do that job. The dangerous confusion is between "please check these two documents" and "here is the shipping instruction, send me a draft later." Both mention a bill of lading. Only the first one is a comparison. We rewrote the prompt against the mailbox until those two stopped collapsing into each other, and until a missing file was escalated instead of marked as a clean match. The label problem (Port of Loading versus Load Port, consignee versus to-

theorder-of) is the same kind of trap. The prompt now states the equivalence rules in the words the scorer uses.

Quota, retries, and a label we should not have saved

The first Vertex attempt sent eight emails as eight simultaneous calls. Google answered with HTTP 429, resource exhausted. Retrying immediately made the burst worse. We stopped that run. The working pattern is the one in section 3: eight emails inside one prompt, three prompts at a time, and a wait when the quota is hit. That finished the inbox with no failed emails.

An earlier Cursor test stored a timeout as if it were a real classification. Email 18 is a request to send a draft bill, with no attachment. The machines timed out twice. The fallback record was a generic operational email, and the desk saved it as success. The prompt is explicit that this message is a document check. The later Vertex pass classified it correctly. A failed call still needs to stay unfinished, so a fallback can never look like an answer. That is not finished, and we are not claiming that it is.

Three writers at once also collided inside Firestore. The audit counter is one document, and parallel saves aborted with “too much contention.” The counter now retries. The emails themselves are separate documents, so the classifications were not lost.

A long job cannot live only inside a Cloud Run request. The service scales to zero, and a background thread dies when the request ends. The 520-email Vertex run wrote into the production database from a process that was allowed to keep running. The website then showed the results. A later version should use a real job, not a thread tied to a page view.

6. Future roadmap

The preliminary desk is the core loop: mail in, classify, compare or escalate, human decision, audit. Turning it into a product is the work after this round, not a replacement of that loop.

- Sign-in and a real workspace. The demo login is one shared clerk. A product needs accounts, a company workspace, and more than one person in the same queue without sharing a password.
- The company mailbox. IMAP with a stored password is enough to prove ingest. The next step is mailbox OAuth, so a team connects the documentation inbox instead of pasting credentials into Settings.
- Do not save a guess. A timed-out model call stays unprocessed and is retried. A mismatch is written only when the two extracted values differ, which would have rejected the false port flag on email 498.
- A job that outlives the page. Classifying a large inbox runs as a Cloud Run job, with the same Firestore the desk reads, and the website only displays progress.
- Learn from the clerk. Validated corrections are the best examples we can add to the prompt. They are already in the audit log.
- Final round. Extend this desk. The public URL, the prompt, the score, and the queue are the base. Authentication, the mailbox, and the stricter save rules are the extension.

The practical value is already visible in the queue. A clerk can see which messages are document checks, open the two values side by side, and leave a record of who resolved them. The credible path beyond the hackathon is to put that queue on the mailbox the team already uses, with real accounts, without changing the decision the model is asked to make.

7. Where the evidence is

- Live desk: <https://zeroday-316196081380.asia-southeast1.run.app>
- Demo clerk: `clerk@zeroday.local` / `clerk123`
- Source and setup: <https://github.com/Dkkama/ZeroDay>
- Prompt: `sdoc_eval/prompts/v1.md`
- Selection score: `sdoc_eval/outputs/flash25_v2/score.json` (final 1.0000)
- The inbox currently loaded in Firestore is the Vertex pass (final 1.0000).

Judges can log in, open Review, and walk a mismatch without running a model call. The decisions are already stored. Settings shows the Vertex provider and will accept a new zip of one email, nine emails, or the full inbox.